

EDGE AI WORKSHOP

RKNN Pipeline Optimization Reference

Hedef Donanım: Rockchip RK3588 (Firefly, Rock 5B, vb.)**Hızlandırıcı Mimari:** 6 TOPS NPU / INT8 Kuantizasyon**Video Çözücü:** Rockchip MPP (Hardware VPU)**Framework:** RKNN-Toolkit2 & GStreamer

Bu doküman, Rockchip **RK3588** tabanlı tek kart bilgisayarlar (SBC) üzerinde gerçek zamanlı, düşük gecikmeli (low-latency) yapay zeka çıkarımı ve nesne takibi (object tracking) gerçekleştirmek amacıyla tasarlanan uçtan uca (end-to-end) Edge AI mimarisini açıklamaktadır.

Sistem genelindeki CPU darboğazlarını (bottleneck) tamamen aşmak adına; asenkron iş parçacıkları (multithreading), donanımsal video kod çözücüler (VPU) ve NPU donanım hızlandırması (RKNN INT8) senkronize edilerek maksimum FPS ve minimum gecikme hedeflenmiştir.

1. Uçtan Uca Mimari (End-to-End Pipeline)

Uygulamanın ana akışı, bellek kopyalamalarını en aza indiren ve her biri birbirinden bağımsız/asekron çalışan 5 farklı birimden (Thread/Süreç) oluşur. Bu mimari sayesinde, kameradan veri transferi ve ön işleme yapılırken yapay zeka işlemcisinin (NPU) boşta beklemesi engellenir.

1. Capture Pipeline (VPU)

IP Camera / USB → GStreamer (mppvideodec) → Hardware Decoding → BGR Frame Queue (maxsize=2, drop=true)



2. Preprocess Thread (CPU)

Read Frame → Letterbox Resize (640x640) → Format: dtype=uint8, NHWC → Preproc Queue (maxsize=4)



3. Inference Thread (NPU)

rknn.inference → RK3588 NPU (INT8 Quantized Model) → Raw Output Tensor → Stage Queue (maxsize=4)



4. Main Thread (CPU)

YOLOv8 Postprocess (NMS / Conf Filter) → BYTETracker (Kalman Filter) → Overlay Drawing → Log Telemetry



5. Streamer Thread (Flask)

Stream Queue (maxsize=2) → cv2.imencode (JPEG Compression) → Web UI / HTTP MJPEG Stream

2. Pipeline Katmanlarının Detayları

2.1. Capture Thread (Görüntü Alma & Donanımsal Decode)

Sistemin girdi katmanıdır. RTSP IP kameralarından veya yerel USB kameralardan gelen video akışlarını toplar:

- **Donanım Hızlandırması:** IP kameralardan gelen sıkıştırılmış H.264/H.265 akışları kesinlikle CPU üzerinde çözülmez. GStreamer hattı içerisindeki `uridecodebin` ve Rockchip Media Process Platform'a ait olan `mppvideodec` eklentileri vasıtasıyla görüntüler doğrudan donanımsal VPU (Video Processing Unit) üzerinde çözülür.

- **Sıfır Gecikme (Zero-Latency) Mantiğı:** `appsink max-buffers=2 drop=true` konfigürasyonu sayesinde, NPU veya Postprocess aşamasında anlık bir yavaşlama yaşansa dahi kuyrukta eski kareler biriktirilmmez; eski kareler doğrudan çöpe atılarak boru hattına daima **en taze karenin (latest frame)** girmesi garanti edilir.

2.2. Preprocess Thread (Ön İşleme Katmanı)

Kameradan alınan ham verilerin, yapay zeka modelinin girdi tensör yapısına uygun matematiksel formata dönüştürüldüğü evredir:

- Giriş çözünürlüğü ne olursa olsun (örn. 1920x1080), modelin girdi boyutu olan `640x640` ölçülerine en-boy oranı (aspect ratio) korunarak **Letterbox** yöntemiyle (boş kalan kenarlar siyaha boyanarak) ölçeklenir.
- RKNN INT8 modelleri, asimetrik kuantizasyon parametrelerini model içerisindeki girdi katmanında barındırdığından, CPU üzerinde ağır `float32` normalizasyon dönüşümlerine gerek kalmaz; veri doğrudan hızlıca `uint8` formatında NPU kuyruğuna aktarılır.

2.3. Inference Thread (NPU & RKNN Çıkarım Motoru)

Sistemin ana işlem çekirdeğidir ve tüm yapay zeka çıkarım yükünü üstlenir:

- **Model Yapısı:** Projede `yo1ov8n_int8.rknn` (YOLOv8 Nano, 8-bit Quantized) modeli koşturulmaktadır.
- **Donanım Optimizasyonu:** RK3588'in sahip olduğu 3 çekirdekli toplam **6 TOPS** gücündeki NPU birimi aktif olarak kullanılır. Model INT8 (8-bit tam sayı) ağırlıklarına sahip olduğu için bellek bant genişliğinden (RAM bandwidth) devasa oranda tasarruf sağlanır ve çıkarım süresi kararlı bir şekilde **10-15 ms** bandına indirgenir.

2.4. Main Thread (Postprocess & Çoklu Nesne Takibi)

Modelin ürettiği ham çıktı tensörlerinin, insan ve sistemlerin anlamlandırabileceği koordinat verilerine dönüştürüldüğü yerdir:

- **Postprocess (NMS):** YOLOv8 çıktı matrisindeki tekrarlanan ve üst üste binen tahmin kutuları, "Non-Maximum Suppression (NMS)" algoritması ile elenerek en yüksek güven skoruna sahip kutuya indirgenir.
- **Takip (BYTETracker):** Yalnızca anlık nesne tespitiyle kalınmaz; arka planda çalışan **Kalman Filtresi** ve IoU (Intersection over Union) eşleştirmesi kullanan BYTETracker mimarisiyle nesnelere benzersiz ID'ler (örn. `Track ID: 12`) atanır ve hareket yönleri ardışık kareler boyunca izlenir.
- **Overlay:** Elde edilen koordinatlar, takip kimlikleri ve anlık donanım/performans metrikleri OpenCV kütüphanesi (`cv2.rectangle` , `cv2.putText`) ile kare üzerine işlenir.

2.5. Streamer Thread (Flask Web Sunucusu)

İşlenen çıktıların ve analiz grafiklerinin ağ üzerindeki istemcilere sunulduğu arabirimdir:

- Görsel çizimleri tamamlanmış olan kare, CPU üzerinde asenkron olarak JPEG formatına sıkıştırılır (`cv2.imencode`).
- Başlatılan gömülü Flask sunucusu (TCP Port 5001) aracılığıyla, bu kareler HTTP protokolü üzerinden standart tarayıcıların doğrudan oynatabildiği **Multipart MJPEG Stream** yapısıyla yayınlanır. Ekstra bir istemci yazılımına ihtiyaç duyulmaz.

3. Optimizasyonlar & Zero-Copy Durumu

Bu proje, Python'ın küresel yorumlayıcı kilidi (GIL) ve sınırları içerisinde yapılabilecek üst düzey çoklu-iş parçacığı (multithreading) mimarisini uygular. Ancak uçtan uca tam anlamıyla bir "True Zero-Copy" (Sıfır Bellek Kopyalama) mimarisine sahip değildir. Donanımsal VPU'dan çıkan bellek adresi (dma_buf) ile NPU girdi tamponları arasındaki geçiş aşamalarında Python katmanında kaçınılmaz olarak bellek kopyalaması gerçekleşmektedir.

Buna karşın, aşağıdaki entegre optimizasyonlar performansı zirvede tutmaktadır:

- **Hardware Video Decoding:** GStreamer + Rockchip MPP entegrasyonu ile CPU dekompresyon yükünün sıfıra indirilmesi.
- **Asymmetric INT8 Quantization:** Model girdi ve çıktılarında veri boyutunun 4 kat küçültülerek NPU cache verimliliğinin artırılması.
- **Ring Buffer & LIFO Dağılımı:** Kuyruklarda `maxsize` limitleri ve `drop=true` parametresiyle biriken gecikmelerin (latency accumulation) mutlak suretle engellenmesi.

4. Donanım Gözlemi (Hardware Telemetry)

Sistem kararlılığını izlemek adına, projenin arka planında Rockchip çekirdek sürücülerinin iç dosya sistemini tarayan bir telemetri mimarisi (`perf_logger_v2.py`) konumlandırılmıştır:

- **NPU Load Analizi:** Her 5 milisaniyede bir kernel seviyesindeki `/sys/kernel/debug/rknpu/load` düğümü taranarak, RK3588'in içindeki 3 ayrı NPU çekirdeğinin (Core0, Core1, Core2) anlık ve tepe (peak) kullanım yüzdeleri loglanır.
- **Güç Tüketimi (Watt):** Kart üzerindeki INA sensörleri ve TCPM gerilim/akım hatlarını barındıran `/sys/class/hwmon/` dizini çapraz sorgulanarak anlık çekilen güç verisi hesaplanır.

5. Sistemi Çalıştırma Talimatı

GStreamer donanım sürücülerine doğrudan erişim sağlamak ve kernel seviyesindeki telemetri dosyalarını (`sysfs/debugfs`) yetki hatası almadan okuyabilmek için betiğin kök yetkileriyle (root) çalıştırılması elzemdir:

```
# GStreamer donanım izinleri ve telemetry loglama yetkileri için sudo kullanımı  
şarttır: sudo .venv/bin/python3 main_v2.py # Alternatif olarak ortamdaki global  
python üzerinden çalıştırma: sudo $(which python3) main_v2.py
```

Önemli Üretim Notu

Endüstriyel canlı ortamlarda (production), Flask MJPEG yayıncısı yerine WebRTC veya RTSP-Server (RStreamer) mimarisine geçilmesi ağ bant genişliği tasarrufu ve kararlılık açısından şiddetle önerilir.